

Hilbert's Hotel

Property-based testing, partição e valor limite aplicados ao kata Hilbert's Hotel

Verificação e Validação — Trabalho T1

PUCRS / Escola Politécnica — 2026/I

Integrantes: Gabriel Bittencourt · Henrique Bueno · Mateus Neubarth

JavaScript · Node 22 · Jest · fast-check

1. Problema escolhido

Kata Hilbert's Hotel — um domínio pequeno com regras finitas e infinitas

O desafio

- Um hotel com infinitos quartos está lotado e chegam novos hóspedes em ônibus.
- A função **hilbertsHotel(rooms, people, buses)** deve atribuir novos quartos sem deixar quartos vazios e sem repetir quartos.
- O domínio combina casos finitos e infinitos e envolve uma fórmula não trivial.

Quatro regras do enunciado

- Pessoas e ônibus finitos → deslocamento por **pessoas × ônibus** quartos.
- Pessoas infinitas + ônibus finitos → quarto × (**ônibus + 1**).
- Ônibus infinitos + pessoas finitas → mesma ideia.
- Ambos infinitos → pareamento diagonal de Cantor: **$n(n+1)/2$** .

Objetivo do estudo: entender quais técnicas detectam problemas que outras não detectam e como medir a eficácia da suíte.

2. Técnicas e base teórica

Três artigos foram usados para orientar as escolhas

Artigo	Contribuição para o trabalho
Corgozinho, Valente e Rocha (2023)	Mostra como PBT aparece na prática: destaque para propriedades de <i>oráculo</i> e <i>roundtrip</i> ; também evidencia categorias e uso de geradores.
Goldstein et al. (2024)	Relata benefícios e custos do PBT na prática: casos de borda, propriedades como documentação e dificuldade de escrever bons geradores/propriedades.
Aichernig e Schumi (2019)	Mostra a derivação de propriedades a partir de modelos de regras de negócio e o uso da especificação como modelo/oráculo.

Escolha: os três trabalhos se complementam — orientam quais propriedades testar, como pensar nos custos/benefícios do PBT e como obter oráculos a partir da especificação.

3. Classes de equivalência + valor limite

Testes por exemplos para organizar o domínio e atingir fronteiras

15

testes de classes de equivalência

11

testes de valor limite

0 / 1

fronteiras críticas

Classes de equivalência

- Particionamento pelas quatro regras do enunciado.
- Casos degenerados: 0 pessoas ou 0 ônibus.
- Lista vazia e entradas inválidas.
- Cada teste identifica a técnica e há rastreabilidade em **tests.md**.

Valor limite

- Fronteiras de pessoas e ônibus em **0/1**.
- Intervalo de quartos que não começa em 1.
- Quarto 0 e quartos não inteiros.
- Limite do inteiro seguro do JavaScript.

4. Property-Based Testing

11 propriedades executadas 300 vezes cada

11 propriedades	300 execuções por propriedade	3.300 execuções de propriedades
---------------------------	---	---

O que foi verificado

- **Oráculo/diferencial:** enumeração explícita dos passageiros e diagonais de Cantor.
- **Invariantes:** injetividade, saída estritamente crescente e função pura.
- **Metamórficas:** equivalências entre diferentes formas de chegada.

Além da fórmula

- Entradas inválidas e exceções esperadas.
- A especificação do enunciado foi usada como modelo/oráculo.
- A abordagem busca combinações difíceis de selecionar manualmente.

Ideia central: em vez de definir apenas exemplos específicos, definimos propriedades que devem permanecer verdadeiras em muitas entradas.

5. Teste de mutação

A suíte foi desafiada com defeitos inseridos manualmente

11

mutantes criados

11/11

mutantes mortos

9

mortos por exemplos + PBT

Defeitos injetados

- Troca de operadores.
- Alteração de constantes.
- Mudança de condições.
- Remoção de validações.

O que a mutação revelou

- Os testes de exemplo, isoladamente, mataram apenas **2** mutantes.
- Esses dois estavam ligados à validação de quarto 0 e ao estouro.
- Os demais **9** foram mortos pela combinação de exemplos e propriedades.
- A mutação expôs um buraco que uma suíte apenas “verde” não mostraria.

6. Resultados

A combinação das técnicas produziu uma suíte forte para o SUT

37

testes totais

100%

cobertura

11/11

mutantes mortos

0

testes falhando

Composição: 15 EP + 11 BV + 11 PBT. Todos os testes passaram.

Principais achados

- PBT detectou erros de fórmula com menor esforço de escolha de entradas.
- Valor limite foi indispensável para valores que os geradores não alcançam.
- A distribuição dos geradores é uma limitação importante.

Interpretação

- As técnicas não competem: elas se complementam.
- Cobertura mede o que foi exercitado no código.
- Mutação ajuda a medir se os testes realmente detectam defeitos.

7. Lições aprendidas e conclusão

O principal resultado foi a complementaridade das técnicas

Lições aprendidas

- Começar por propriedades de **alta alavancagem**: oráculo, invariantes e metamórficas.
- Medir eficácia com **cobertura e mutação** é tão importante quanto escrever testes.
- Quando a especificação é executável, ela pode ajudar a construir o oráculo.
- Geradores precisam ser avaliados: passar nos testes não garante que exploraram os limites relevantes.

Limitações

- Oráculos e solução foram escritos pelo mesmo grupo e podem compartilhar uma interpretação equivocada.
- Os 11 mutantes foram escolhidos manualmente.
- Execuções PBT usam sementes aleatórias.
- O SUT é pequeno e puramente funcional; os resultados não generalizam diretamente para sistemas com estado/efeitos colaterais.

Conclusão: classes de equivalência organizam o espaço de entrada; valor limite cobre fronteiras; PBT explora combinações e propriedades globais; mutação avalia a capacidade real de detecção da suíte.

Trabalho futuro: ampliar os geradores para valores próximos aos limites e utilizar uma ferramenta de mutação automática.

Referências principais

Fontes utilizadas na resenha crítica

- AICHERNIG, Bernhard K.; SCHUMI, Richard. *Property-based testing of web services by deriving properties from business-rule models*. Software & Systems Modeling, v. 18, p. 889–911, 2019.
- CORGOZINHO, Arthur Lisboa; VALENTE, Marco Tulio; ROCHA, Henrique. *How developers implement property-based tests*. ICSME, 2023.
- GOLDSTEIN, Harrison et al. *Property-based testing in practice*. ICSE, 2024.
- AZART___. *Hilbert's Hotel*. Codewars, 2025.